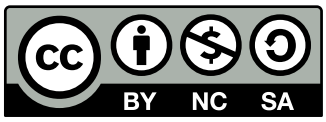
Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

Cross-language JSON parser interoperability

Oula Kivalo

Master's Thesis
2025



Aalto University
School of Science

Author Oula Kivalo

Title Cross-language JSON parser interoperability

Degree programme Computer, Communication and Information Sciences

Major Security and Cloud Computing

Supervisor Prof. TODO

Advisor TODO

Date TODO **Number of pages** 17+6 **Language** English

Abstract
TODO

Keywords JSON parsing, JSON interoperability, Cross-language parsing

Contents

Symbols and abbreviations

Symbols

P	set of parsers
S	set of parsing results

Abbreviations

JSON	JavaScript object notation
IETF	internet engineering task force
RLE	Run-length encoding

Strings

TODO - Not sure where to place this

Unless otherwise specified, strings in this paper represent a character sequence encoded in UTF-8. Strings are surrounded by single quotation marks "`'`" which are not included in them. Example: `'{"key": "value"}'`.

There are a few character sequences which have a special meaning and are encoded following different rules:

Sequence	Description	Encoding
`\n`	New line character	`0A` in hexadecimal
`\t`	Horizontal tab	`09` in hexadecimal
`\r`	Carriage return	`0D` in hexadecimal
`\x`a	Single byte. a is as a two digit hexadecimal number with a range of $a \in \{0, 1, ..., 255\}$	Single byte with a value of a

1 Introduction

Comments are colored in orange.
This thesis is currently in its initial stages and will be rewritten. As such, I'm looking for feedback on its general structure, and not on its text.
This chapter is mostly copied from the thesis proposal

JavaScript Object Notation(JSON) has become the de facto standard for data interchange on the internet, and as such, multiple parsers have been developed for it across programming languages. With the rise of microservice-based cloud architectures, where small independent services often utilize JSON for communication, these different parser implementations may be used to parse the same inputs. If a malicious actor is able to inject JSON data into such communication, inconsistencies in parser behavior may enable privilege escalation, denial-of-service attacks, or other vulnerabilities.

Even with its minimalistic design, JSON parsing can introduce security vulnerabilities due to ambiguities in the specification and incorrect parser implementations. Due to its apparent simplicity and minimalistic featureset, JSON can easily be missed as a potential attack vector.

1.1 Problem statement

Despite the widespread use of JSON, academic research on JSON parser interoperability is surprisingly limited. Most prior work has been carried out by independent security researchers, though their findings are already several years old and may be outdated. This thesis aims to further research in the area by documenting interoperability concerns across different parser implementations in multiple programming languages.

This thesis builds on the work of Möller et al., who showed that JSON parsers have large amounts of discrepancies between them. This is done by investigating a larger set of parsers and placing a focus on major discrepancies between them which may lead to vulnerabilities.

This thesis tests a collection of 40 parsers across ten languages.

TODO - update parser count

1.2 Structure of the thesis

TODO

2 Background

2.1 JavaScript object notation

TODO - Introduction to JSON and its specification

2.1.1 History of JSON

TODO - Brief overview of how we got here. Historical baggage in JSON

2.1.2 Ambiguities in the JSON specification

- Duplicate key precedence
- Character encoding
- String comparison
- Extremely large numbers
- Nesting depth
- Version numbering, or lack thereof

2.2 Fuzz testing

TODO - What fuzz testing is and what can be achieved using it.

2.2.1 Mutation based fuzz testing

TODO - Mutating hand-picked test cases in different ways.

2.2.2 Black-box fuzzing

Not sure whether automatic test case generation will be implemented. I have some plans on how I could implement this for black-box fuzzing, but this might be out of scope for the thesis

2.2.3 Cross-language fuzz testing

TODO - Testing something across multiple programming languages and seeing how results differ with the same test case.

2.2.4 Previous work in JSON fuzz testing

Current fuzz testing is mostly focused on individual parsers and rarely includes other parsers. Popular parsers commonly have a test suite which contains edge cases and fuzz testing.

2.3 Run-length encoding

Run-length encoding (RLE) is a lossless form of data compression where consecutive identical occurences of data are stored as a single occurence of the data and a count of its consecutive occurences. It has an extremely low overhead and provides good compression ratios for data which contains repeated values often.

2.4 Vulnerabilities

Will be updated depending on findings and related work

2.4.1 Denial of service

2.4.2 Remote code execution

3 Methods

3.1 Parser selection

The parsers were selected from json.org and GitHub based on their popularity.

3.2 Testing parsers

Parsers were tested using a custom fuzzing application.

3.3 Test case selection

Test cases used for mutation-based fuzz testing were selected from edge cases shown in the JSON specification and from the test suites of popular parsers.

3.4 Analyzing results

The results gained from fuzzing were stored for later analyzis.

3.5 Searching vulnerabilities

Using the results gained from fuzzing, open source projects were investigated for potential vulnerabilities.

4 Implementation

4.1 Tooling

Due to the large number of dependencies used, everything was implemented inside a Docker container running in a Linux environment. The programs developed for the thesis were written in multiple different programming languages:

1. Bash
2. C++
3. Clojure
4. Golang
5. JavaScript
6. Lua
7. PHP
8. Python
9. Rust

The fuzzer and analyzer were written in Rust due to the large amounts of data processed. The project uses multiple helper scripts written in Bash. Other languages used were needed for the JSON parsers.

4.1.1 Libraries used

TODO - an ungodly amount. Need to condense this section to fit within the page target count for a master's thesis. I also refer to different JSON parsers by a custom codename. Need to put a list of them here

4.2 Test server

TODO - Discuss fuzzing implementation

Test server generates test cases and shares them with clients via TCP sockets. Due to the high performance requirements, the server was written using Rust.

4.3 Clients

Clients contain multiple JSON parsers which parse the incoming data sent by the test server. Each JSON parser has their own client instance which connects to the test server.

Some clients support multiple languages. The client for C++ also contains parsers for C. The client for Clojure also contains parsers for Java.

4.4 Server-client communication

A custom binary protocol over TCP was developed for the thesis.

4.5 Result compression and decompression

The server saved the parsing results for later analysis. Due to the total size of the parsing results being over a petabyte, an efficient compression solution was needed. After existing solutions were tested and their performance was deemed unsatisfactory, a custom solution based on run-length encoding (RLE) was developed.

The set of parsing results S contains the following elements:

$$S \in \{(p, d, r) \mid p \in P, d, r \in \{0, 1\}^*\} \tag{1}$$

where P is the set of parsers, d is binary representation of JSON data, and r is the parsing result parser p produced for input d.

TODO - Explain how most of this information can be stripped during compression, run-length encoding (RLE), the binary data format used, and how it can be decompressed.

4.6 Analysis

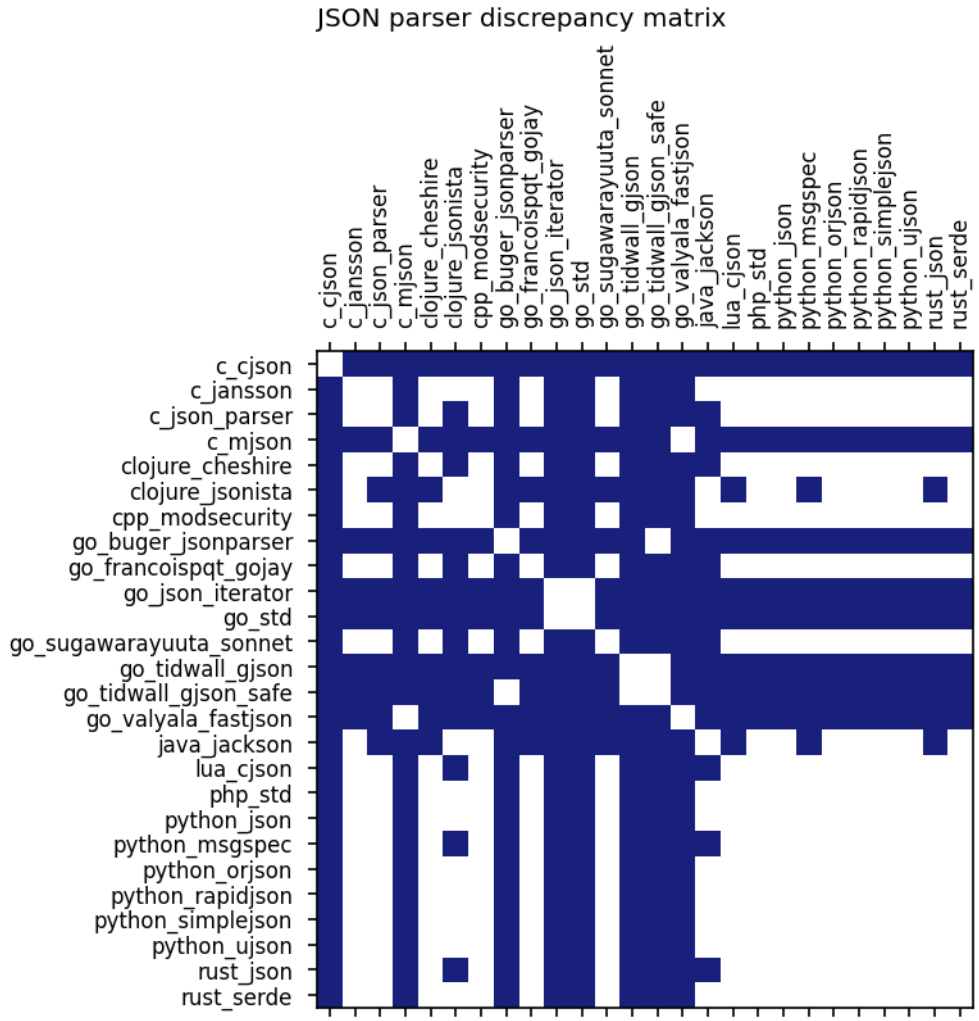


Figure 1: JSON parsing mismatches. A cell is colored in blue if the parsers in the corresponding column and row can retrieve different values when querying the same key in the same JSON object.

5 Results

5.1 JSON parsing inconsistencies

5.1.1 Different value access in JSON objects

TODO - Main contribution

JSON parsers can retrieve different values from the same object when querying for the same key as seen in Figure 1 and table A.

5.2 Impact

5.2.1 Schema bypass in Tyk

Copied from gitlab with slight modifications

Tyk is an API gateway which can be used to access multiple internal API's through a single external access point. It can be configured from a single OpenAPI document which provides information on how an API is structured and how it can be interacted with. Additionally, Tyk can be used to validate requests and responses going through it based on schemas defined in the OpenAPI document.

Hypothetical scenario A login API is accessible thgough Tyk as shown in Figure 1. Tyk validates login information using JSON schemas contained in an OpenAPI document. In this scenario, the API developer believes the login information to be validated and passes it unsafely to an SQL query. Due to Golang standard library JSON parser being case-insensitive, the schema validation can be bypassed by passing additional capitalized key-value pairs to the API.

5.2.2 Rate limiting bypass in HAProxy

Copied from gitlab with slight modifications

HAProxy is a popular reverse proxy with over a billion downloads in Docker Hub described as "a free, very fast and reliable reverse-proxy offering high availability, load balancing, and proxying for TCP and HTTP-based applications". One of its features is to act as a rate limiter between a client and a server. As an example, it can be configured to rate-limit requests by dropping those that match a certain IP address, header, URL, or body parameter after too many have been received within a timespan.

Internally, HAProxy uses mjson to parse JSON data inside request bodies and headers. When encountering duplicate keys, mjson uses only the first duplicate key while ignoring the rest. While this functionality is correct according to the latest JSON specification, it differs from the industry standard of using the last duplicate key inside JSON objects. Additionally, msjon does not parse unicode encoded values in strings.

Due to these discrepancies, the JSON object `'{"rol\u0065":"admin","role":"user","role":"admin"}'` gets parsed differently by mjson and the vast majority of other JSON parsers. While msjon believes that the key role has a value of user, almost all other parsers believe the value to be admin. This enables us to affect how HAProxy routes traffic when it is dependent on JSON values.

Hypothetical scenario An API is behind HAProxy, which combines multiple different API's to a single externally accessible endpoint. One of these API's handles user logins. To prevent password guessing and brute forcing, HAProxy limits the number of login attempts for a single username from the same IP address. If the number of login attempts exceeds five within a 60 second window, the proxy denies additional

Due to the API only accepting JSON data inside the POST request, HAProxy gets the username used for the rate limiting from JSON data. Using the discrepancies described above, we can formulate the login request as `'{"usernam\u0065":"<actual>", "username":"<haproxy>", "username":"<actual>", "password":"<password>"}'`. In this way, HAProxy and the API disagree on what the requested username is. Thus we can bypass the rate limiting by sending random information in the second field, while sending the actual username in the first and third fields.

6 Evaluation

6.1 Limitations

6.2 Results

7 Conclusions

References

TODO

A JSON parsing mismatches

P1	P2	JSON	P1["q"]	P2["q"]
c_cjson	c_jansson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	c_json_parser	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	c_mjson	`'{"\u0071":2,"q":3}'`	'2'	'3'
c_cjson	clojure_cheshire	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	clojure_jsonista	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	cpp_modsecurity	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	go_buger_jsonparser	`'{"q\u0000":2,"q":3}'`	'2'	'3'
c_cjson	go_francoispqt_gojay	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	go_json_iterator	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	go_std	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	go_sugawarayuuta_sonnet	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	go_tidwall_gjson	`'{"q\u0000":2,"q":3}'`	'2'	'3'
c_cjson	go_tidwall_gjson_safe	`'{"q\u0000":2,"q":3}'`	'2'	'3'
c_cjson	go_valyala_fastjson	`'{"\u0071":2,"q":3}'`	'2'	'3'
c_cjson	java_jackson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	lua_cjson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	php_std	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	python_json	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	python_msgspec	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	python_orjson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	python_rapidjson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	python_simplejson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	python_ujson	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	rust_json	`'{"q":2,"q":3}'`	'2'	'3'
c_cjson	rust_serde	`'{"q":2,"q":3}'`	'2'	'3'
c_jansson	c_cjson	`'{"q":2,"q":3}'`	'2'	'3'
c_jansson	c_mjson	`'{"q":3,"q":2}'`	'2'	'3'
c_jansson	go_buger_jsonparser	`'{"q":3,"q":2}'`	'2'	'3'
c_jansson	go_json_iterator	`'{"q":2,"Q":3}'`	'2'	'3'
c_jansson	go_std	`'{"q":2,"Q":3}'`	'2'	'3'
c_jansson	go_tidwall_gjson	`'{"q":3,"q":2}'`	'2'	'3'
c_jansson	go_tidwall_gjson_safe	`'{"q":3,"q":2}'`	'2'	'3'
c_jansson	go_valyala_fastjson	`'{"q":3,"q":2}'`	'2'	'3'
c_json_parser	c_cjson	`'{"q":2,"q":3}'`	'2'	'3'
c_json_parser	c_mjson	`'{"q":3,"q":2}'`	'2'	'3'
c_json_parser	clojure_jsonista	`'{"q":2,"\xffq":3}'`	'2'	'3'
c_json_parser	go_buger_jsonparser	`'{"q":3,"q":2}'`	'2'	'3'
c_json_parser	go_json_iterator	`'{"q":2,"Q":3}'`	'2'	'3'
c_json_parser	go_std	`'{"q":2,"Q":3}'`	'2'	'3'
c_json_parser	go_tidwall_gjson	`'{"q":3,"q":2}'`	'2'	'3'
c_json_parser	go_tidwall_gjson_safe	`'{"q":3,"q":2}'`	'2'	'3'
c_json_parser	go_valyala_fastjson	`'{"q":3,"q":2}'`	'2'	'3'
c_json_parser	java_jackson	`'{"q":2,"\xffq":3}'`	'2'	'3'
c_mjson	c_cjson	`'{"\u0071":2,"q":3}'`	'2'	'3'
c_mjson	c_jansson	`'{"q":3,"q":2}'`	'2'	'3'
c_mjson	c_json_parser	`'{"q":3,"q":2}'`	'2'	'3'
c_mjson	clojure_cheshire	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	clojure_jsonista	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	cpp_modsecurity	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	go_buger_jsonparser	`'{"\u0071":2,"q":3}'`	'3'	'2'
c_mjson	go_francoispqt_gojay	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	go_json_iterator	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	go_std	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	go_sugawarayuuta_sonnet	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	go_tidwall_gjson	`'{"q\q":2,"q":3}'`	'3'	'2'
c_mjson	go_tidwall_gjson_safe	`'{"\u0071":2,"q":3}'`	'3'	'2'
c_mjson	java_jackson	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	lua_cjson	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	php_std	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	python_json	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	python_msgspec	`'{"q":2,"q":3}'`	'2'	'3'
c_mjson	python_orjson	`'{"q":2,"q":3}'`	'2'	'3'

c_mjson	python_rapidjson	'{"q":2,"q":3}'	'2'	'3'
c_mjson	python_simplejson	'{"q":2,"q":3}'	'2'	'3'
c_mjson	python_ujson	'{"q":2,"q":3}'	'2'	'3'
c_mjson	rust_json	'{"q":2,"q":3}'	'2'	'3'
c_mjson	rust_serde	'{"q":2,"q":3}'	'2'	'3'
clojure_cheshire	c_cjson	'{"q":2,"q":3}'	'2'	'3'
clojure_cheshire	c_mjson	'{"q":2,"q":3}'	'2'	'3'
clojure_cheshire	clojure_jsonista	'{"q":2,"\xffq":3}'	'2'	'3'
clojure_cheshire	go_buger_jsonparser	'{"q":3,"q":2}'	'2'	'3'
clojure_cheshire	go_json_iterator	'{"q":2,"Q":3}'	'2'	'3'
clojure_cheshire	go_std	'{"q":2,"Q":3}'	'2'	'3'
clojure_cheshire	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
clojure_cheshire	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
clojure_cheshire	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
clojure_cheshire	java_jackson	'{"q":2,"\xffq":3}'	'2'	'3'
clojure_jsonista	c_cjson	'{"q":2,"q":3}'	'2'	'3'
clojure_jsonista	c_json_parser	'{"q":2,"\xffq":3}'	'2'	'3'
clojure_jsonista	c_mjson	'{"q":2,"q":3}'	'2'	'3'
clojure_jsonista	clojure_cheshire	'{"q":2,"\xffq":3}'	'2'	'3'
clojure_jsonista	go_buger_jsonparser	'{"q":3,"q":2}'	'2'	'3'
clojure_jsonista	go_francoispqt_gojay	'{"q":2,"\xffq":3}'	'3'	'2'
clojure_jsonista	go_json_iterator	'{"q":2,"Q":3}'	'2'	'3'
clojure_jsonista	go_std	'{"q":2,"Q":3}'	'2'	'3'
clojure_jsonista	go_sugawarayuuta_sonnet	'{"q":2,"\xffq":3}'	'3'	'2'
clojure_jsonista	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
clojure_jsonista	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
clojure_jsonista	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
clojure_jsonista	lua_cjson	'{"q":2,"\xffq":3}'	'3'	'2'
clojure_jsonista	python_msgspec	'{"q":2,"\xffq":3}'	'3'	'2'
clojure_jsonista	rust_json	'{"q":2,"\xffq":3}'	'3'	'2'
cpp_modsecurity	c_cjson	'{"q":2,"q":3}'	'2'	'3'
cpp_modsecurity	c_mjson	'{"q":2,"q":3}'	'2'	'3'
cpp_modsecurity	go_buger_jsonparser	'{"q":3,"q":2}'	'2'	'3'
cpp_modsecurity	go_json_iterator	'{"q":2,"Q":3}'	'2'	'3'
cpp_modsecurity	go_std	'{"q":2,"Q":3}'	'2'	'3'
cpp_modsecurity	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
cpp_modsecurity	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
cpp_modsecurity	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
go_buger_jsonparser	c_cjson	'{"q\u0000":2,"q":3}'	'2'	'3'
go_buger_jsonparser	c_jansson	'{"q":3,"q":2}'	'2'	'3'
go_buger_jsonparser	c_json_parser	'{"q":3,"q":2}'	'2'	'3'
go_buger_jsonparser	c_mjson	'{"\u0071":2,"q":3}'	'3'	'2'
go_buger_jsonparser	clojure_cheshire	'{"q":3,"q":2}'	'2'	'3'
go_buger_jsonparser	clojure_jsonista	'{"q":3,"q":2}'	'2'	'3'
go_buger_jsonparser	cpp_modsecurity	'{"q":3,"q":2}'	'2'	'3'
go_buger_jsonparser	go_francoispqt_gojay	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	go_json_iterator	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	go_std	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	go_sugawarayuuta_sonnet	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	go_tidwall_gjson	'{"q" 2,"q":3}'	'3'	'2'
go_buger_jsonparser	go_valyala_fastjson	'{"\u0071":2,"q":3}'	'2'	'3'
go_buger_jsonparser	java_jackson	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	lua_cjson	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	php_std	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	python_json	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	python_msgspec	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	python_orjson	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	python_rapidjson	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	python_simplejson	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	python_ujson	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	rust_json	'{"q":2,"q":3}'	'2'	'3'
go_buger_jsonparser	rust_serde	'{"q":2,"q":3}'	'2'	'3'
go_francoispqt_gojay	c_cjson	'{"q":2,"q":3}'	'2'	'3'
go_francoispqt_gojay	c_mjson	'{"q":2,"q":3}'	'2'	'3'
go_francoispqt_gojay	clojure_jsonista	'{"q":2,"\xffq":3}'	'3'	'2'
go_francoispqt_gojay	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
go_francoispqt_gojay	go_json_iterator	'{"q":2,"Q":3}'	'2'	'3'
go_francoispqt_gojay	go_std	'{"q":2,"Q":3}'	'2'	'3'

go_francoispqt_gojay	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
go_francoispqt_gojay	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
go_francoispqt_gojay	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
go_francoispqt_gojay	java_jackson	'{"q":2,"\xffq":3}'	'2'	'3'
go_json_iterator	c_cjson	'{"q":2,"q":3}'	'2'	'3'
go_json_iterator	c_jansson	'{"q":2,"Q":3}'	'2'	'3'
go_json_iterator	c_json_parser	'{"q":2,"Q":3}'	'2'	'3'
go_json_iterator	c_mjson	'{"q":2,"q":3}'	'2'	'3'
go_json_iterator	clojure_cheshire	'{"q":2,"Q":3}'	'2'	'3'
go_json_iterator	clojure_jsonista	'{"q":2,"Q":3}'	'2'	'3'
go_json_iterator	cpp_modsecurity	'{"q":2,"Q":3}'	'2'	'3'
go_json_iterator	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
go_json_iterator	go_francoispqt_gojay	'{"q":2,"Q":3}'	'2'	'3'
go_json_iterator	go_sugawarayuuta_sonnet	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
go_json_iterator	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
go_json_iterator	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
go_json_iterator	java_jackson	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	lua_cjson	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	php_std	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	python_json	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	python_msgspec	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	python_orjson	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	python_rapidjson	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	python_simplejson	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	python_ujson	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	rust_json	'{"q":2,"Q":3}'	'3'	'2'
go_json_iterator	rust_serde	'{"q":2,"Q":3}'	'3'	'2'
go_std	c_cjson	'{"q":2,"q":3}'	'2'	'3'
go_std	c_jansson	'{"q":2,"Q":3}'	'2'	'3'
go_std	c_json_parser	'{"q":2,"Q":3}'	'2'	'3'
go_std	c_mjson	'{"q":2,"q":3}'	'2'	'3'
go_std	clojure_cheshire	'{"q":2,"Q":3}'	'2'	'3'
go_std	clojure_jsonista	'{"q":2,"Q":3}'	'2'	'3'
go_std	cpp_modsecurity	'{"q":2,"Q":3}'	'2'	'3'
go_std	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
go_std	go_francoispqt_gojay	'{"q":2,"Q":3}'	'2'	'3'
go_std	go_sugawarayuuta_sonnet	'{"q":2,"Q":3}'	'3'	'2'
go_std	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
go_std	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
go_std	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
go_std	java_jackson	'{"q":2,"Q":3}'	'3'	'2'
go_std	lua_cjson	'{"q":2,"Q":3}'	'3'	'2'
go_std	php_std	'{"q":2,"Q":3}'	'3'	'2'
go_std	python_json	'{"q":2,"Q":3}'	'3'	'2'
go_std	python_msgspec	'{"q":2,"Q":3}'	'3'	'2'
go_std	python_orjson	'{"q":2,"Q":3}'	'3'	'2'
go_std	python_rapidjson	'{"q":2,"Q":3}'	'3'	'2'
go_std	python_simplejson	'{"q":2,"Q":3}'	'3'	'2'
go_std	python_ujson	'{"q":2,"Q":3}'	'3'	'2'
go_std	rust_json	'{"q":2,"Q":3}'	'3'	'2'
go_std	rust_serde	'{"q":2,"Q":3}'	'3'	'2'
go_sugawarayuuta_sonnet	c_cjson	'{"q":2,"q":3}'	'2'	'3'
go_sugawarayuuta_sonnet	c_mjson	'{"q":2,"q":3}'	'2'	'3'
go_sugawarayuuta_sonnet	clojure_jsonista	'{"q":2,"\xffq":3}'	'3'	'2'
go_sugawarayuuta_sonnet	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
go_sugawarayuuta_sonnet	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
go_sugawarayuuta_sonnet	go_std	'{"q":2,"Q":3}'	'3'	'2'
go_sugawarayuuta_sonnet	go_tidwall_gjson	'{"q":3,"q":2}'	'2'	'3'
go_sugawarayuuta_sonnet	go_tidwall_gjson_safe	'{"q":3,"q":2}'	'2'	'3'
go_sugawarayuuta_sonnet	go_valyala_fastjson	'{"q":3,"q":2}'	'2'	'3'
go_sugawarayuuta_sonnet	java_jackson	'{"q":2,"\xffq":3}'	'2'	'3'
go_tidwall_gjson	c_cjson	'{"q\u0000":2,"q":3}'	'2'	'3'
go_tidwall_gjson	c_jansson	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	c_json_parser	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	c_mjson	'{"q\q":2,"q":3}'	'3'	'2'
go_tidwall_gjson	clojure_cheshire	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	clojure_jsonista	'{"q":3,"q":2}'	'2'	'3'

go_tidwall_gjson	cpp_modsecurity	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	go_buger_jsonparser	'{"q" 2,"q":3}'	'3'	'2'
go_tidwall_gjson	go_francoispqt_gojay	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	go_json_iterator	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	go_std	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	go_sugawarayuuta_sonnet	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson	go_valyala_fastjson	'{"q\q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	java_jackson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	lua_cjson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	php_std	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	python_json	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	python_msgspec	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	python_orjson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	python_rapidjson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	python_simplejson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	python_ujson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	rust_json	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson	rust_serde	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	c_cjson	'{"q\u0000":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	c_jansson	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	c_json_parser	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	c_mjson	'{"\u0071":2,"q":3}'	'3'	'2'
go_tidwall_gjson_safe	clojure_cheshire	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	clojure_jsonista	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	cpp_modsecurity	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	go_francoispqt_gojay	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	go_json_iterator	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	go_std	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	go_sugawarayuuta_sonnet	'{"q":3,"q":2}'	'2'	'3'
go_tidwall_gjson_safe	go_valyala_fastjson	'{"\u0071":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	java_jackson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	lua_cjson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	php_std	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	python_json	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	python_msgspec	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	python_orjson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	python_rapidjson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	python_simplejson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	python_ujson	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	rust_json	'{"q":2,"q":3}'	'2'	'3'
go_tidwall_gjson_safe	rust_serde	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	c_cjson	'{"\u0071":2,"q":3}'	'2'	'3'
go_valyala_fastjson	c_jansson	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	c_json_parser	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	clojure_cheshire	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	clojure_jsonista	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	cpp_modsecurity	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	go_buger_jsonparser	'{"\u0071":2,"q":3}'	'2'	'3'
go_valyala_fastjson	go_francoispqt_gojay	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	go_json_iterator	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	go_std	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	go_sugawarayuuta_sonnet	'{"q":3,"q":2}'	'2'	'3'
go_valyala_fastjson	go_tidwall_gjson	'{"q\q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	go_tidwall_gjson_safe	'{"\u0071":2,"q":3}'	'2'	'3'
go_valyala_fastjson	java_jackson	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	lua_cjson	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	php_std	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	python_json	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	python_msgspec	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	python_orjson	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	python_rapidjson	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	python_simplejson	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	python_ujson	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	rust_json	'{"q":2,"q":3}'	'2'	'3'
go_valyala_fastjson	rust_serde	'{"q":2,"q":3}'	'2'	'3'
java_jackson	c_cjson	'{"q":2,"q":3}'	'2'	'3'
java_jackson	c_json_parser	'{"q":2,"\xffq":3}'	'2'	'3'
java_jackson	c_mjson	'{"q":2,"q":3}'	'2'	'3'

java_jackson	clojure_cheshire	'{"q":2,"\xffq":3}'	'2'	'3'
java_jackson	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
java_jackson	go_francoispqt_gojay	'{"q":2,"\xffq":3}'	'2'	'3'
java_jackson	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
java_jackson	go_std	'{"q":2,"Q":3}'	'3'	'2'
java_jackson	go_sugawarayuuta_sonnet	'{"q":2,"\xffq":3}'	'2'	'3'
java_jackson	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
java_jackson	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
java_jackson	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
java_jackson	lua_cjson	'{"q":2,"\xffq":3}'	'3'	'2'
java_jackson	python_msgspec	'{"q":2,"\xffq":3}'	'3'	'2'
java_jackson	rust_json	'{"q":2,"\xffq":3}'	'3'	'2'
lua_cjson	c_cjson	'{"q":2,"q":3}'	'2'	'3'
lua_cjson	c_mjson	'{"q":2,"q":3}'	'2'	'3'
lua_cjson	clojure_jsonista	'{"q":2,"\xffq":3}'	'3'	'2'
lua_cjson	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
lua_cjson	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
lua_cjson	go_std	'{"q":2,"Q":3}'	'3'	'2'
lua_cjson	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
lua_cjson	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
lua_cjson	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
lua_cjson	java_jackson	'{"q":2,"\xffq":3}'	'3'	'2'
php_std	c_cjson	'{"q":2,"q":3}'	'2'	'3'
php_std	c_mjson	'{"q":2,"q":3}'	'2'	'3'
php_std	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
php_std	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
php_std	go_std	'{"q":2,"Q":3}'	'3'	'2'
php_std	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
php_std	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
php_std	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
python_json	c_cjson	'{"q":2,"q":3}'	'2'	'3'
python_json	c_mjson	'{"q":2,"q":3}'	'2'	'3'
python_json	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
python_json	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
python_json	go_std	'{"q":2,"Q":3}'	'3'	'2'
python_json	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
python_json	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
python_json	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	c_cjson	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	c_mjson	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	clojure_jsonista	'{"q":2,"\xffq":3}'	'3'	'2'
python_msgspec	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
python_msgspec	go_std	'{"q":2,"Q":3}'	'3'	'2'
python_msgspec	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
python_msgspec	java_jackson	'{"q":2,"\xffq":3}'	'3'	'2'
python_orjson	c_cjson	'{"q":2,"q":3}'	'2'	'3'
python_orjson	c_mjson	'{"q":2,"q":3}'	'2'	'3'
python_orjson	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
python_orjson	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
python_orjson	go_std	'{"q":2,"Q":3}'	'3'	'2'
python_orjson	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
python_orjson	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
python_orjson	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
python_rapidjson	c_cjson	'{"q":2,"q":3}'	'2'	'3'
python_rapidjson	c_mjson	'{"q":2,"q":3}'	'2'	'3'
python_rapidjson	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
python_rapidjson	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
python_rapidjson	go_std	'{"q":2,"Q":3}'	'3'	'2'
python_rapidjson	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
python_rapidjson	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
python_rapidjson	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
python_simplejson	c_cjson	'{"q":2,"q":3}'	'2'	'3'
python_simplejson	c_mjson	'{"q":2,"q":3}'	'2'	'3'
python_simplejson	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
python_simplejson	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'

python_simplejson	go_std	'{"q":2,"Q":3}'	'3'	'2'
python_simplejson	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
python_simplejson	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
python_simplejson	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
python_ujson	c_cjson	'{"q":2,"q":3}'	'2'	'3'
python_ujson	c_mjson	'{"q":2,"q":3}'	'2'	'3'
python_ujson	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
python_ujson	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
python_ujson	go_std	'{"q":2,"Q":3}'	'3'	'2'
python_ujson	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
python_ujson	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
python_ujson	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
rust_json	c_cjson	'{"q":2,"q":3}'	'2'	'3'
rust_json	c_mjson	'{"q":2,"q":3}'	'2'	'3'
rust_json	clojure_jsonista	'{"q":2,"\xffq":3}'	'3'	'2'
rust_json	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
rust_json	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
rust_json	go_std	'{"q":2,"Q":3}'	'3'	'2'
rust_json	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
rust_json	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
rust_json	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'
rust_json	java_jackson	'{"q":2,"\xffq":3}'	'3'	'2'
rust_serde	c_cjson	'{"q":2,"q":3}'	'2'	'3'
rust_serde	c_mjson	'{"q":2,"q":3}'	'2'	'3'
rust_serde	go_buger_jsonparser	'{"q":2,"q":3}'	'2'	'3'
rust_serde	go_json_iterator	'{"q":2,"Q":3}'	'3'	'2'
rust_serde	go_std	'{"q":2,"Q":3}'	'3'	'2'
rust_serde	go_tidwall_gjson	'{"q":2,"q":3}'	'2'	'3'
rust_serde	go_tidwall_gjson_safe	'{"q":2,"q":3}'	'2'	'3'
rust_serde	go_valyala_fastjson	'{"q":2,"q":3}'	'2'	'3'